Command Craft Tutorial

웹 브라우저에서 Git 명령어를 게임처럼 배우자

Team CCT · 강희전 이승재 황상하 · 2026.04.30

```
➜  ~  git clone CCT
```

💡 목차

1.프로젝트 소개

2.기술 스택

3.시스템 아키텍처

4.주요 기능

5.채점 시스템 & Git 시뮬레이터

6.DB 설계

6.DB 설계

7.브랜치 전략

8.역할분담

9.진행사항

10.어려웠던 점 & 해결 방법

11.아쉬운 점 & 추가하고 싶은 기능

💡 프로젝트 소개

❌ 기존 학습 방법의 문제점

- 책/문서로만 배우면 실습 부족
- 실제 터미널은 실수 시 되돌리기 어려움
- 반복 학습 환경이 없음
- 재미없어서 금방 포기

Git은 개발자라면 반드시 알아야 하는 도구이지만 처음 배울 때 명령어가 낯설고 어렵습니다.

✅ CCT의 해결책

- 웹 브라우저에서 안전하게 실습
- 스테이지별 단계적 학습
- 게임처럼 재미있는 구성
- 즉각적인 성공/실패 피드백

실제 터미널 환경 없이도 Git 명령어를 체계적으로 배울 수 있습니다.

⚙️ 기술 스택

Frontend

React 18 · Vite · React Router DOM · Axios · Zustand

Backend

Node.js · Express · Knex.js · dotenv · nodemon

Database

PostgreSQL · Supabase · migrations · seeds

협업 도구

GitHub · Jira · feature 브랜치 전략 · PR & Code Review

시스템 아키텍처

API 목록

GET /stages

GET /stages/:id

POST /stages/:id/submit

POST /users/login

PATCH /users/:id/progress

사용자 (브라우저)

React 프론트엔드 :5173

Express 백엔드 :3000

PostgreSQL (Supabase 클라우드)

🎮 주요 기능

🔐 로그인

닉네임 입력 후 시작
진행상황 자동 저장

📋 스테이지 목록

전체 스테이지 한눈에
완료/진행/잠금 표시

💻 터미널 UI

VS Code 스타일 터미널
실제처럼 명령어 입력

✅ 채점 시스템

정확 일치 + 정규식
Git 시뮬레이션 엔진

💡 힌트 모달

틀렸을 때 힌트 팝업
스테이지 이동 시 초기화

🏆 완료 화면

모든 스테이지 클리어
최종 점수 & 다시 시작

⚡ 채점 시스템 & Git 시뮬레이터

📝 채점 방식

1. **정확 일치 채점**
   입력: `git init`
   정답: `git init` → ✅

2. **정규식 채점**
   입력: `git commit -m "hello"`
   정규식: `^git commit -m ".+"$`
   → ✅ (메시지 자유롭게)

🔧 Git 시뮬레이션 엔진

유저별 가상 Git 상태 메모리 유지

상태 관리 항목:

- initialized (초기화 여부)
- stagedFiles (스테이지 파일)
- commits (커밋 히스토리)
- branch (현재 브랜치)

순서 틀리면 에러 출력:

❌ `fatal: not a git repository`

🗄️ DB 설계

stages 테이블

컬럼명	타입
id	integer (PK)
title	string
description	string
mission	string
answer	string
answer_regex	string
hint	string
difficulty	string
created_at	datetime

users 테이블

컬럼명	타입
id	integer (PK)
username	string
current_stage	integer
score	integer
created_at	datetime

☁️ Supabase 클라우드 PostgreSQL

팀원 모두 같은 DB 공유

.env로 연결 정보 관리

🌿 브랜치 전략

main — 최종 제출본

develop — 통합 브랜치

- feature/backend
- feature/frontend
- feature/stage-clear
- feature/stage-list
- feature/hint-modal
- feature/animations

feature 브랜치 → PR 생성 → 코드 리뷰 → develop merge → main merge

👥 역할 분담

백엔드 + 프론트 : 강희전

- Express API 설계 및 구현
- Supabase DB 연결
- Git 시뮬레이션 엔진
- 터미널 UI 구현
- 게임 화면 전체 개발
- 힌트 모달 & 애니메이션

DB 데이터 : 황상하

- 스테이지 데이터 작성
- 난이도별 구성 (기초/중급/심화)
- seeds 파일 관리
- migrations 설계

UI/UX 디자인 : 이승재

- 전체 컬러 테마 통일
- 로그인 페이지 디자인
- 게임 화면 디자인
- 컴포넌트 스타일 정리

진행사항

UI 핵심 라이브러리 결정

Unity 기반 앱 인터페이스

- 장점
  - 게임적 UX
  - 시각적 연출
- 단점
  - 프로젝트 성격과 차이
  - 구현 복잡도
  - 앱 배포

배포 과정 등의 어려움이 있을 것으로 판단

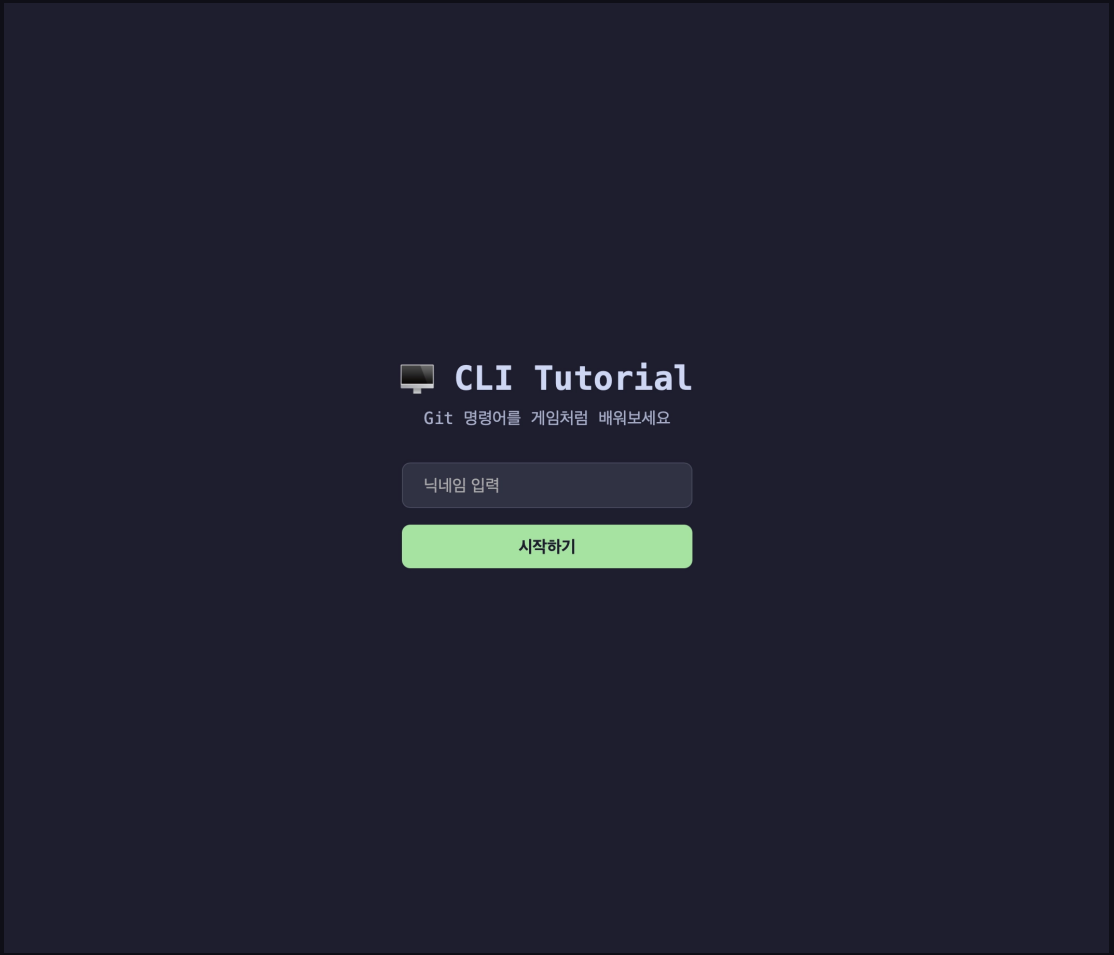
React 기반 웹 인터페이스

- 장점
  - 프로젝트 적합성
  - 빠른 개발
  - 배포 편의
- 단점
  - 그래픽 연출 제한
  - 게임적 UX

프로젝트의 성격과 React가 더 잘 맞을 것으로 판단

진행사항

로그인 페이지 생성



진행사항

스테이지 목록

스테이지 목록

스테이지를 선택해서 시작하세요

01 저장소 초기화 기초
`git init` 을 입력하세요
진행중 ▶

02 파일 추가하기 기초
`git add .` 을 입력하세요
잠금

03 커밋 만들기 기초
`git commit -m "메시지"` 형식으로 입력하세요
잠금

04 브랜치 만들기 중급
`git branch 브랜치이름` 을 입력하세요
잠금

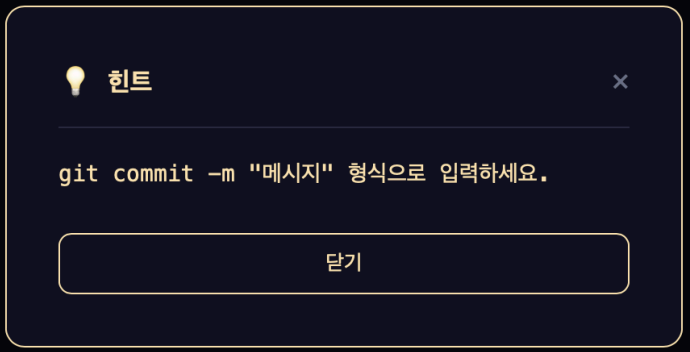
05 브랜치 이동 중급
`git checkout 브랜치이름` 을 입력하세요
잠금

진행사항

스테이지 완료 페이지

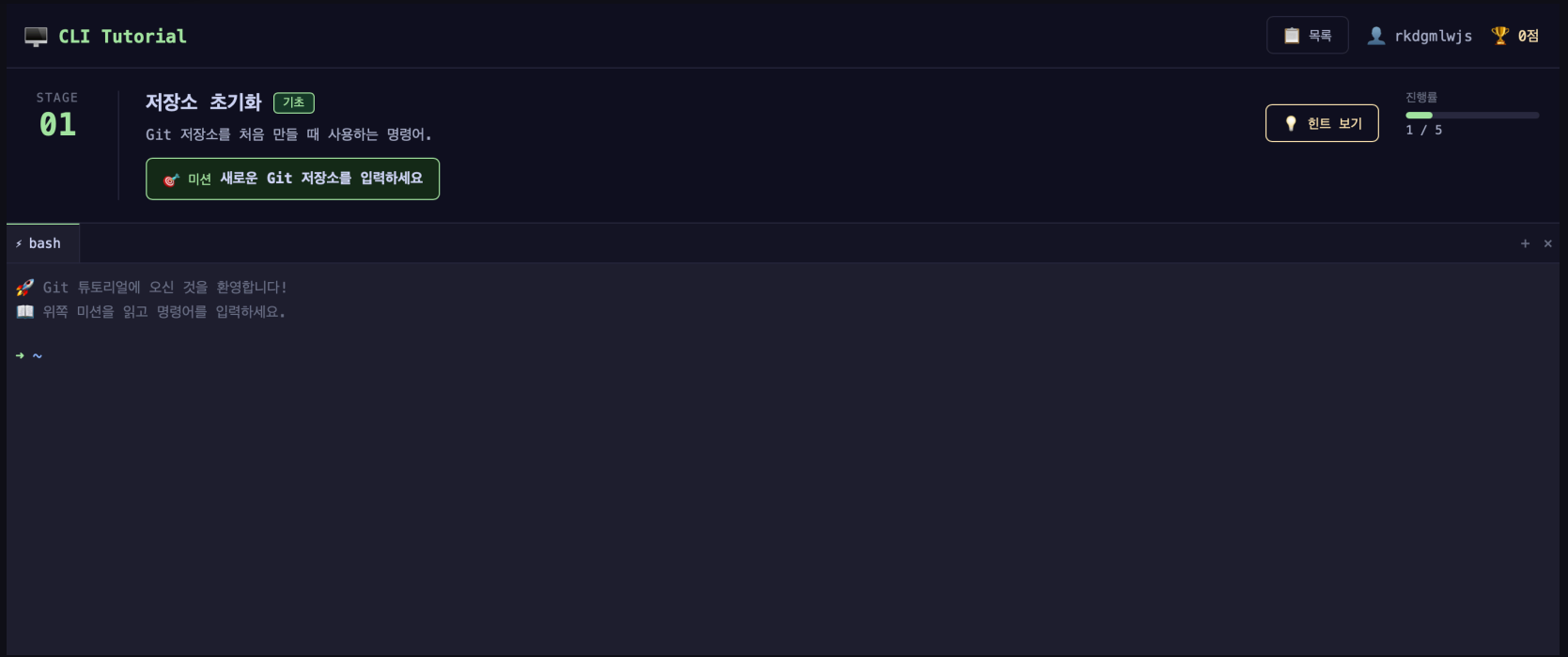
진행사항

힌트 팝업 구현



진행사항

게임 화면 구현


CLI Tutorial
목록
rkdgmlwjs
0점
STAGE
01
저장소 초기화
기초
Git 저장소를 처음 만들 때 사용하는 명령어.
미션 새로운 Git 저장소를 입력하세요
힌트 보기
진행률
1 / 5
bash
Git 튜토리얼에 오신 것을 환영합니다!
위쪽 미션을 읽고 명령어를 입력하세요.
→ ~

🔥 어려웠던 점 & 해결 방법

❌ xterm.js 렌더러 에러

this._renderer.value.dimensions 에러 발생

→ xterm.js 대신 React로 직접 터미널 UI 구현

❌ Supabase IPv4 연결 문제

Direct connection IPv4 미지원으로 연결 실패

→ Transaction pooler 방식으로 변경하여 해결

❌ 명령어 채점 유연성

git commit 메시지가 달라도 정답 처리 필요

→ 정규식 기반 채점 시스템 도입으로 해결

❌ 팀 브랜치 전략

팀 프로젝트 첫 경험으로 충돌 발생

→ main/develop/feature 브랜치 전략 수립

📝 아쉬운 점 & 추가하고 싶은 기능

😅 아쉬운 점

- 스테이지 수가 적음 (5개)
- 모바일 최적화 미흡
- 실제 Git 환경과 완전히 동일하지 않음
- 로그인 시스템이 단순함 (닉네임만)
- 배포 환경 미구축

🚀 추가하고 싶은 기능

- 타이머 & 목숨 시스템
- 리더보드 (점수 순위표)
- Linux 명령어 스테이지 추가
- 폭죽 애니메이션 (confetti)
- 소셜 로그인
- 실제 배포 (Vercel + Railway)

Command Craft Tutorial

Git 명령어를 게임처럼 배울 수 있는

인터랙티브 웹 튜토리얼

감사합니다 🙏

github.com/CommandCraftTutorial/CCT